Data Science Assessment – Task 1 Report

Overview
This document addresses **Task 1** of the Data Science assessment.

1. *Time series challenge*: Predict the weekly number of expected service requests per hex that will be created each week using sr_hex.csv, for 4 weeks past the end of the dataset.

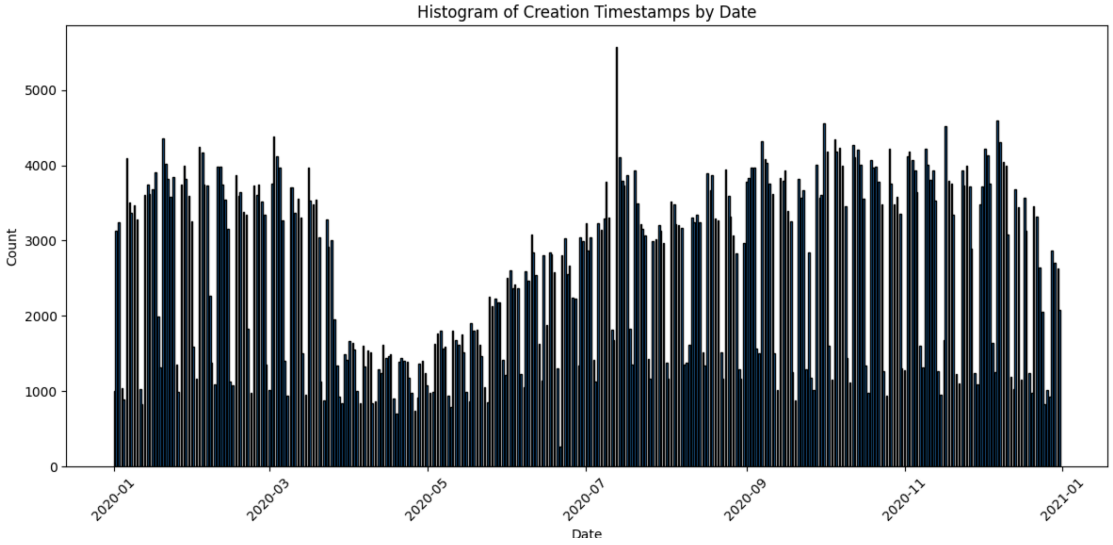
Data Preparation and Preprocessing
Python code: *DataScienceTask1_Preprocessing.py*

- This script loads the raw event records CSV - *sr.csv.gz*.
- We visualise the data to understand more about its make-up along with possible temporal trends.
- This script attributes the geographical hex data to the count dataframe from *city-hex-polygons-8.geojson.*
  Note: this attribution is not used, as the sr_hex.csv.gz dataset with attribution already completed is used for model development.
- Each record includes:
  - a timestamp
  - a hex_index identifier.

Preprocessing Workflow:

1. **Convert Timestamps**
   - creation_timestamp is converted to datetime format.
2. **Aggregate by Week**
   - Each timestamp is mapped to the start of its week (week_start).
   - Counts of events per hex per week are calculated.
3. **Pivot to Wide Format**
   - Data is reshaped so:
     - Rows = unique hex_index
     - Columns = counts for each week.
   - Missing weeks are filled with zeros.

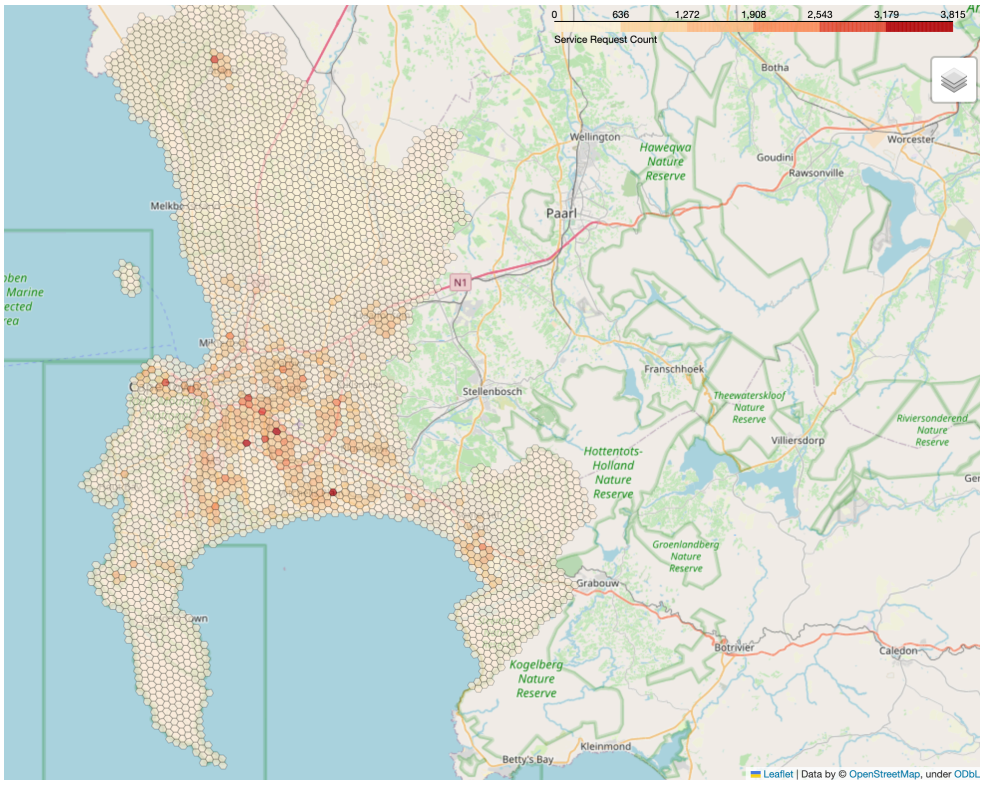
Basic data visualisations:



Above is a daily breakdown of service calls. There is an obvious weekly and daily trend visible.

Output from preprocessing:

- A spatial / temporal pivoted DataFrame (pivoted) used in subsequent modelling steps.
- Visualizations are generated to show spatial distributions and counts across hexes.



Above image highlights the spatial layout of service calls over the entire year.

Prediction Objective
Goal:
Predict the event counts for each hex **four weeks into the future**.
Train/Test Split:

- **Training Set:** 80% of hexes
- **Test Set:** 20% of hexes

Initial thought:
We have one year's worth of data. Therefore, we should be able to capture some of the temporal trends visual. However, possessing only one year presents a challenge, as the trend for year-end (whenever the data ends) to year start (whenever is starts the previous year) is not captured. We will not loop the data in this case, such that the model takes the final weeks as input and predicts the beginning weeks, as this presents discontinuous data. Hexes will be treated independently in this prediction exercise without exogenous feature (other than LSTM_seasonal). Meaning, trends from one hex will not be represented in another. This

approach ensures that spatial dependencies do not confound the model's ability to learn purely temporal patterns within each individual hex.

Modelling Approaches
We trained four models ranging in complexity to forecast counts before selecting the best performing model:

1. SARIMAX Model (*SARIMAX_prediction.py*)
Approach:
- **Input:**
  - For each hex, historical weekly counts *excluding* the last 4 weeks.
  - **Example**: if 53 weeks total, the first 49 weeks are training.
- **No exogenous regressors included.**

Modelling Details:
- A separate SARIMAX model is fitted to each hex time series.
- Counts are modelled as a linear function of past observations.

---

2. Random Forest Regressor (*RandomForestRegression.py*)
Approach:
- Single model trained across all hexes.

Input:
- For each hex, rolling windows of lagged counts.
  - **Example**: a lag window of 13 weeks provides 13 input features.
- No external regressors.

Modelling Details:
- Random Forest Regressor trained to predict the count 4 weeks ahead.
- Learns nonlinear relationships in lagged counts.

Output:
- Predicted counts for each hex in the test set.
- Compared against actual counts.

---

3. LSTM Regressor (*LSTM_PredModel.py*)
Approach:
- Single model trained across all hexes.

Input:
- Rolling windows of lagged weekly counts.

Modelling Details:
- Loss function = Mean Square Error
- LSTM neural network learns temporal dependencies.

Output:
- Predicted counts 4 weeks ahead.

---

4. LSTM with Seasonal Features (*LSTM_seasonal.py*)
Approach:
- Same as the standard LSTM above, however includes:
  - Sine and cosine encodings of week numbers to capture seasonality.
  - Due to multiple feature scale, inputs are normalise and rescaled back to count scale.

Model Performance Summary

Model	Input Window (weeks)	MAE	RMSE	R^2	MAPE
SARIMAX	49	2.3682	4.5821	0.6033	63.33%
Random Forest Regression	4	3.1654	6.3667	0.6350	77.13%
	13	2.8726	5.5582	0.7009	71.70%
	26	2.7900	5.5177	0.7497	62.48%
	48	2.7646	6.1100	0.6074	63.59%
LSTM	4	3.1109	5.9851	0.6191	74.20%
	13	2.2572	5.2863	0.7028	62.60%
	26	**2.2050**	**5.1519**	**0.7178**	**59.38%**
	48	3.6031	6.8040	0.5077	113.65%
LSTM + Seasonal Features	4	2.6511	5.6269	0.6633	64.00%
	13	2.9466	5.4652	0.6824	73.84%
	26	2.9359	5.8973	0.6302	66.78%
	48	3.5432	5.6061	0.6658	92.54%

Best Model:

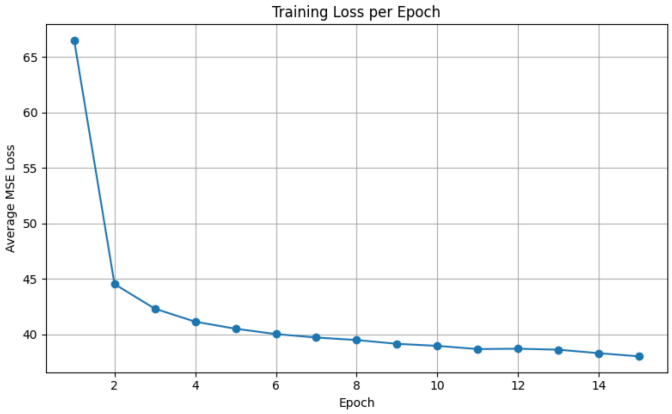
- **LSTM (26-week historical window) –** i.e., the model considers ½ year of input data to understand temporal trends for prediction.
    - **MAE:** 2.2050
    - **RMSE:** 5.1519
    - **R^2:** 0.7178
    - **MAPE:** 59.38%

Model Interpretation

LSTM Model Configuration:

- Hidden size: 64
- Layers: 2

Training convergence plot:



Metrics:

MAE (Mean Absolute Error):

On average, the model's predictions were approximately 2 counts away from the actual

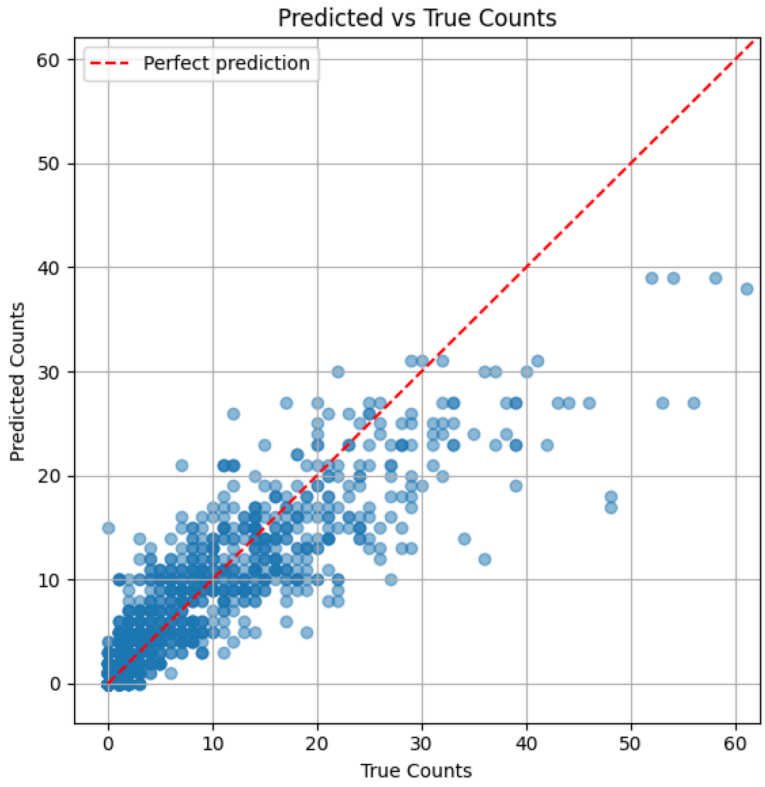
observed counts. This indicates reasonably low absolute error in terms of counts, suggesting that for most time steps and locations, the model captures the central tendency of the data fairly well.

RMSE (Root Mean Squared Error):
The RMSE was about 5 counts, which is more than double the MAE. Because RMSE penalises larger errors more heavily than MAE, this suggests that while many predictions are close to actual counts, there are some instances where the model's errors spike (e.g., under- or over-predictions during peaks or sudden drops). The discrepancy between MAE and RMSE highlights the presence of occasional higher-magnitude deviations.

R^2 (Coefficient of Determination):
An R^2 of approximately 72% indicates that the model explains about 72% of the variance in the observed counts. This is generally considered moderate to strong explanatory power, meaning the model captures much of the signal in the data but still leaves around 28% of the variability unexplained. This residual variability could be due to unmodeled factors (e.g., exogenous events, spatial dependencies) or inherent randomness in the counts.

MAPE (Mean Absolute Percentage Error):
The MAPE of about 59% indicates that, on average, prediction errors were roughly 59% of the true counts. This relatively high percentage error is especially important because it reflects the scale-sensitivity of MAPE: when true counts are small (e.g., counts close to zero), even small absolute errors become large percentage errors. This is consistent with time series involving low counts, where predicting 2 instead of 1 already yields a 100% error.

Graphical representation of results:



Final Predictions

- After identifying the best model, it was retrained on the **full dataset** (no train/test split).
- Predicted counts for each hex for the next 4 weeks.
- Results saved to ***predictions.csv.***

Excerpt from final model predictions.

hex_index	week_1_pred	week_2_pred	week_3_pred	week_4_pred
88ad361229fffff	16.0	15.0	15.0	15.0
88ad36122bfffff	40.0	40.0	41.0	40.0
88ad36122dfffff	14.0	14.0	14.0	15.0
88ad361231fffff	25.0	26.0	27.0	27.0
88ad361233fffff	8.0	8.0	8.0	8.0
88ad361235fffff	39.0	38.0	38.0	37.0
88ad361237fffff	21.0	22.0	22.0	22.0
88ad361239fffff	10.0	10.0	10.0	10.0
88ad36123bfffff	1.0	1.0	1.0	1.0
88ad36123dfffff	25.0	24.0	25.0	25.0
88ad361241fffff	13.0	13.0	13.0	13.0